



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

**FinanceFlow: Plataforma Web de Gestión
Financiera e Inteligencia Artificial para el Control
de Gastos Personales**

Curso: Construcción de Software I

Docente: Mag. Ricardo Eduardo Valcárcel Alvarado

Integrantes:

Sebastian Arce Bracamonte

(2019062886)

Brant Antony Chata Choque

(2020067577)

Tacna – Perú

2026

Documento de Estándares de Programación - FinanceFlow

Este documento define las convenciones y estándares de código para el proyecto **FinanceFlow**. Todos los desarrolladores deben adherirse a estas pautas para garantizar consistencia, mantenibilidad y calidad en el sistema.

1. Convenciones de Nomenclatura de Archivos y Carpetas

Regla:

- **/frontend**: PascalCase para componentes (`MovimientoForm.jsx`), camelCase para utilidades/hooks (`useAuth.js`).
- **/backend**: kebab-case o notación de punto para su función (`movimientos.controller.js`, `usuario.model.js`).
- **/ml_backend**: snake_case estricto (`train_model.py`, `predict_category.py`).
- **/sdk**: kebab-case (`auth-adapter.js`, `component-migration.js`).

Correcto

JavaScript

```
// frontend/src/components/ReportChart.jsx
// backend/controllers/logs.controller.js
// ml_backend/data_processor.py
```

✗ Incorrecto:

JavaScript

```
// frontend/src/components/report_chart.jsx ✗
// backend/controllers/LogsController.js ✗
// ml_backend/DataProcessor.py ✗
```

2. Estándares de JavaScript/React

Regla: Usar componentes funcionales, desestructuración de `props`, y mantener un orden de importaciones estricto (1. React/Externos, 2. Componentes Internos, 3. Estilos/Assets).

Correcto:

JavaScript

```
import React, { useState } from 'react';
import { useForm } from 'react-hook-form';
import MovimientoItem from './MovimientoItem';

const MovimientoForm = ({ onSubmit, isLoading }) => {
  const { register, handleSubmit } = useForm();

  return (
    <form onSubmit={handleSubmit(onSubmit)} className="p-4
bg-white rounded-lg shadow">
      { /* ... */ }
    </form>
  );
};
export default MovimientoForm;
```

✗ Incorrecto:

JavaScript

```
import MovimientoItem from './MovimientoItem';
import React from 'react'; // ✗ Orden incorrecto

const MovimientoForm = (props) => { // ✗ Falta desestructurar
  return <form className="form">{props.children}</form>;
};
```

3. Estándares de Node.js/Express

Regla: Separar la lógica de negocio en controladores. Las rutas solo deben mapear endpoints a funciones de controlador. Pasar siempre los errores al middleware global usando `next(error)`.

Correcto:

JavaScript

```
// routes/movimientos.js
router.get('/', MovimientosController.getAll);

// controllers/movimientos.controller.js
const getAll = async (req, res, next) => {
  try {
    const movimientos = await
MovimientoService.findAll(req.user.id);
    res.status(200).json({ success: true, data: movimientos });
  } catch (error) {
    next(error);
  }
};
```

✗ Incorrecto:

JavaScript

```
// routes/movimientos.js
router.get('/', async (req, res) => { // ✗ Poner lógica en rutas
  const movs = await db.Movimiento.find();
  res.send(movs); // ✗ Falta try/catch
});
```

4. Estándares de Python/FastAPI (Microservicio AI)

Regla: Usar `snake_case` para variables y funciones. Es obligatorio el uso de *Type Hints* de Python y modelos Pydantic para validación de requests/responses.

Correcto:

```
JavaScript
from fastapi import FastAPI
from pydantic import BaseModel

class ReceiptInput(BaseModel):
    image_url: str

@app.post("/api/v1/ocr/extract", response_model=dict)
def extract_receipt_data(payload: ReceiptInput) -> dict:
    text: str = perform_ocr(payload.image_url)
    return {"success": True, "extracted_text": text}
```

✗ Incorrecto:

```
JavaScript
@app.post("/ocr")
def ExtractReceipt(payload): # ✗ PascalCase, sin Typehints
    t = do_ocr(payload["url"])
    return {"text": t}
```

5. Estándares de MongoDB/Mongoose

Regla: Nombres de modelos en `PascalCase` y singular. Los esquemas deben usar validaciones integradas de Mongoose (`required`, `enum`, `min`, `max`).

Correcto:

JavaScript

```
// database/movimiento.model.js
const mongoose = require('mongoose');

const movimientoSchema = new mongoose.Schema({
  tipo: { type: String, enum: ['ingreso', 'egreso'], required: true },
  monto: { type: Number, required: true, min: [0.01, 'Monto inválido'] },
}, { timestamps: true });

module.exports = mongoose.model('Movimiento', movimientoSchema);
```

✗ Incorrecto:

JavaScript

```
const Movimientos = new Schema({ // ✗ Nombre en plural
  tipo: String, // ✗ Faltan validaciones
  monto: Number
});
module.exports = mongoose.model('movimientos', Movimientos); // ✗ minúscula y plural
```

6. Estándares de API REST

Regla: Usar sustantivos plurales para URLs. Retornar los códigos HTTP correctos (200, 201, 400, 404, 500) y usar un *envelope* (envoltura) estándar { success: boolean, data: any, error: string|null }.

Correcto:

```
JavaScript
// Request
GET /api/v1/movimientos

// Response (200 OK)
{
  "success": true,
  "data": [{ "id": 1, "monto": 100 }],
  "error": null
}
```

✗ Incorrecto:

```
JavaScript
// Request
GET /api/v1/getMovimientos // ✗ Verbos en la URL

// Response (200 OK, pero con error lógico) // ✗
{
  "status": "error",
  "message": "No encontrado"
}
```

7. Estándares de Manejo JWT

Regla: Los tokens JWT deben enviarse en el header `Authorization: Bearer <token>`. El token de acceso debe tener expiración corta (p.ej. 15m - 1h).

Correcto:

```
JavaScript
// frontend/src/services/api.js
const token = localStorage.getItem('token'); // 0 idealmente en memoria si hay HttpOnly refresh token
axios.get('/api/movimientos', {
  headers: { Authorization: `Bearer ${token}` }
});
```

✗ Incorrecto:

```
JavaScript
// ✗ Enviar el token en la URL o body
axios.post('/api/movimientos', { token: myToken, monto: 100 });
```

8. Estándares de Git

Regla: Usar prefijos descriptivos para ramas (`feature/`, `bugfix/`, `hotfix/`). Mensajes de commit según *Conventional Commits*.

Correcto:

```
Shell
# Rama
git checkout -b feature/auth-adapter

# Commit
git commit -m "feat(sdk): add auth adapter implementation with fallback"
```


✗ Incorrecto:

Shell

```
git checkout -b mi-rama-miguel # ✗ Falta prefijo
git commit -m "arregla el error de login" # ✗ Falta conventional
commit
```

9. Estándares de Manejo de Errores

Regla: En Node.js, usar middlewares globales de manejo de error en vez de responder directamente en el catch. Todos los logs de error deben estructurarse usando p.ej. Pino/Winston en producción.

Correcto:

JavaScript

```
// middlewares/errorHandler.js
const errorHandler = (err, req, res, next) => {
  console.error(`[ERROR] ${err.message}`);
  const status = err.statusCode || 500;
  res.status(status).json({ success: false, data: null, error:
err.message });
};
```

✗ Incorrecto:

JavaScript

```
catch (error) {
  console.log(error); // ✗ Log sucio
  res.send('Ocurrió un error'); // ✗ No envuelve en JSON, status
por defecto 200
}
```

10. Estándares de Testing (Jest)

Regla: Archivos terminados en `.test.js`. Agrupar casos de uso con `describe` y nombrar las aserciones con `it()` de forma leíble comenzando en minúscula. Mantener mínimo 80% de coverage.

Correcto:

JavaScript

```
// backend/__tests__/movimientos.controller.test.js
describe('Movimientos Controller', () => {
  it('should return 400 if validation fails', async () => {
    const res = await request(app).post('/movimientos').send({});
    expect(res.statusCode).toBe(400);
    expect(res.body.success).toBe(false);
  });
});
```

✗ Incorrecto:

JavaScript

```
// ✗ Archivo tester_mov.js
test('testea error', async () => { // ✗ Sin bloque describe,
  // ...                             nombre poco descriptivo
  expect(res.statusCode).toEqual(400);
});
```